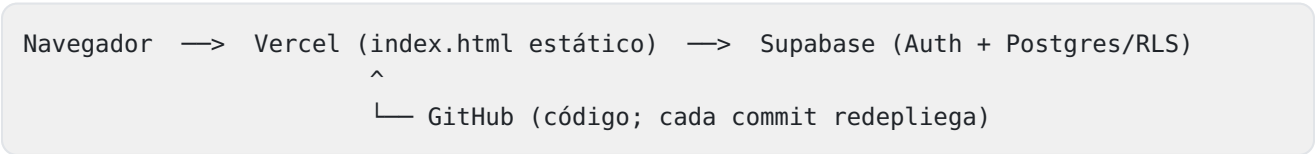


# Kaizen — Documentación técnica y de despliegue

Guía para montar Kaizen desde cero: **GitHub** (código) → **Vercel** (hosting) → **Supabase** (datos + login). Está pensada para que alguien pueda reproducir el despliegue siguiendo los pasos en orden.

## 1. Arquitectura general

- **App de una sola página:** todo el front está en un único archivo HTML ( `index.html` ), sin paso de compilación. No hay framework ni backend propio.
- **Capa de almacenamiento ( `Store` ):** se autodetecta y funciona en tres modos:
- **localStorage** — si se abre el HTML sin configurar nada (datos solo en ese navegador).
- **Supabase** — si hay `config.js` con credenciales y sesión iniciada (nube + login). Es el modo de producción.
- **Cowork** — si corre dentro de ese entorno.
- **Hosting:** Vercel sirve el sitio y **redespliega en cada commit** del repo de GitHub.
- **Datos y autenticación:** Supabase (Postgres + Auth), con seguridad por fila (**RLS**).



## 2. Archivos del proyecto (carpeta `iteraciones-app/`)

| Archivo                              | Para qué sirve                                                                                               | ¿Se sube al repo?                |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------|
| <code>index.html</code>              | La app publicada (copia de <code>iteraciones.html</code> con los scripts inyectados)                         | <b>Sí</b>                        |
| <code>config.js</code>               | URL y clave pública (anon) de Supabase                                                                       | <b>Sí</b>                        |
| <code>supabase-schema.sql</code>     | Tablas base: <code>app_state</code> y <code>signup_requests</code>                                           | No (solo se ejecuta en Supabase) |
| <code>shared-games-schema.sql</code> | <code>profiles</code> (@usuario), <code>shared_games</code> , <code>game_invites</code> y función de aceptar | No                               |
| <code>admins-schema.sql</code>       | Tabla <code>admins</code> y restricción de solicitudes al administrador                                      | No                               |
| <code>leave-game-fix.sql</code>      |                                                                                                              | No                               |

| Archivo                                                                | Para qué sirve                                       | ¿Se sube al repo? |
|------------------------------------------------------------------------|------------------------------------------------------|-------------------|
|                                                                        | Función para salir de un proyecto compartido         |                   |
| <code>backup-app-state.sql</code>                                      | Copia de seguridad puntual de <code>app_state</code> | No                |
| <code>MANUAL-USUARIO.md</code> / <code>DOCUMENTACION-TECNICA.md</code> | Documentación                                        | Opcional          |

El archivo de trabajo del diseño es `iteraciones.html` (en la raíz del proyecto local).  
**Nunca se sube tal cual:** hay que generar `index.html` a partir de él (ver sección 7).

### 3. Paso 1 — GitHub (código)

1. Crea una cuenta en <https://github.com> y un repositorio **público** (p. ej. `Iteraciones`).
2. Sube por la web (**Add file → Upload files**) el contenido de `iteraciones-app/`: como mínimo `index.html` y `config.js`. Los `.sql` no hacen falta en el repo (se ejecutan en Supabase).
3. No subas la carpeta `.git`.

Regla práctica: en GitHub solo tocas archivos dentro de esta carpeta de despliegue.

### 4. Paso 2 — Vercel (hosting)

1. Crea cuenta en <https://vercel.com> e **importa el repositorio** de GitHub.
2. **Framework Preset = Other** (es estático). Deploy.
3. Obtendrás una URL del tipo `https://TU-PROYECTO.vercel.app` (en este caso, `kaizen-idm.vercel.app`).
4. A partir de ahí, **cada commit** en GitHub redepliega solo.

Si el repo no aparece al importar: en Vercel, "Adjust GitHub App Permissions" y da acceso al repo.

## 5. Paso 3 — Supabase (datos + login)

### 5.1 Crear proyecto

1. Crea cuenta en <https://supabase.com> → **New project** (guarda la contraseña de la base de datos).

### 5.2 Ejecutar el SQL (en este orden)

En **SQL Editor** → **New query**, pega y ejecuta cada archivo **en orden**:

1. **supabase-schema.sql** — crea **app\_state** (datos por usuario) y **signup\_requests** (solicitudes de acceso) con sus políticas RLS.
2. **shared-games-schema.sql** — crea **profiles** (directorio con **@usuario** único), añade la columna **username** a **signup\_requests**, y crea **shared\_games**, **game\_invites** y la función **accept\_game\_invite**.
3. **admins-schema.sql** — crea **admins** y restringe las solicitudes de acceso a administradores. **Edita el correo del insert por el tuyo** antes de ejecutar.
4. **leave-game-fix.sql** — crea la función **leave\_shared\_game** (salir de un compartido de forma segura).
5. **announcements-schema.sql** — crea **announcements** (mensajes del admin a todos los usuarios). Requiere **admins** ya creada.
6. **webhook-trigger.sql** — (opcional, para el aviso por email) disparador que llama a la Edge Function al recibir una solicitud. Ver sección 12.

Si Supabase avisa de "destructive" o de habilitar RLS al crear tablas, es normal (es por crear tablas/activar RLS). Elige **Run and enable RLS** cuando lo ofrezca.

### 5.3 Conectar la app con Supabase ( **config.js** )

1. En Supabase: **Project Settings** → **API** (o *Data API*). Copia el **Project URL** y la **anon public key**.
2. Rellena **config.js** y súbelo a GitHub: 

```
js window.SUPABASE_CONFIG = { url: "https://TUPROYECTO0.supabase.co", anonKey: "eyJhbGciOiI..."};
```

 - El **url** debe ser **https://xxxx.supabase.co** (no la URL del panel). Un **url** mal puesto da el error "Invalid path specified in request URL".

### 5.4 Configurar el login

- **Authentication** → **URL Configuration**: pon en **Site URL** y en **Redirect URLs** la URL de Vercel ( <https://kaizen-idm.vercel.app> y [https://kaizen-idm.vercel.app/\\*\\*](https://kaizen-idm.vercel.app/**) ). Esto hace que los correos (invitación, validación, recuperación) apunten a Kaizen y no a una URL antigua.
- **Authentication** → **Sign In / Providers** → **Email**: deja **Email** habilitado. Para que al crear usuarios entren con contraseña sin fricción, puedes desactivar "Confirm email" según prefieras.

- **Cierra los registros públicos:** desactiva "**Allow new users to sign up**". El alta se hace solo por el flujo de solicitudes + creación manual (ver 6.1).

---

## 6. Flujos clave

---

### 6.1 Alta de usuarios (aprobación manual, sin servidor)

1. La persona rellena **Solicitar acceso** → se guarda una fila en `signup_requests` (no se crea cuenta).
2. El **administrador** (correo en la tabla `admins`) la ve en el **buzón** ✉ y la **aprueba** (se marca y copia el correo).
3. El administrador pulsa **Aprobar** en el buzón → la Edge Function `approve-signup` invita al usuario automáticamente (le llega el correo para fijar contraseña) y marca la solicitud como *Aprobada*. Si el correo ya existía, le envía un enlace de restablecer. Ver sección 13.
4. Al entrar por primera vez, la persona elige su **@usuario** (se guarda en `profiles`; se pre-rellena con el que pidió al registrarse).

El aviso por email al administrador cuando llega una solicitud está en la sección 12.

### 6.2 Compartir un juego

1. El dueño comparte: el juego se **migra** de su `app_state` a la tabla `shared_games` (dueño = él, `members` vacío).
2. Invita por **@usuario** → se crea una fila en `game_invites` (pendiente). La búsqueda usa el directorio `profiles` (nunca expone correos).
3. El invitado **acepta** → la función `accept_game_invite` lo añade a `members`.
4. Ambos editan; cada guardado actualiza `shared_games` con `updated_by` / `updated_by_name` y un **contador** `rev` para **control de conflictos** (si cambió desde que abriste, se avisa y recarga).
5. Salir: la función `leave_shared_game` quita al usuario de `members`. Dejar de compartir (dueño): borra la fila (el juego vuelve a ser privado).

### 6.3 Recuperar contraseña

"¿No recuerdas tu contraseña?" → `resetPasswordForEmail` → al volver del enlace, la app detecta el evento de recuperación y pide la nueva contraseña (`updateUser`).

---

## 7. Mantenimiento: publicar cambios del código

El diseño se edita en `iteraciones.html` (archivo de trabajo). Para publicar:

1. Regenera `index.html` a partir de `iteraciones.html` **inyectando dos scripts antes de `</head>`**: - `https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2` - `config.js` (Es la única diferencia entre `iteraciones.html` y `index.html`.)
2. Sube el `index.html` resultante a GitHub (carpeta de despliegue) → Vercel redepliega solo.

Publicar código **no toca la base de datos**: los datos viven en Supabase, no en el HTML.

## 8. Modelo de datos (resumen)

| Tabla                        | Columnas principales                                                                                                                                                               | Acceso (RLS)                                  |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| <code>app_state</code>       | <code>user_id</code> (pk), <code>data</code> (jsonb), <code>updated_at</code>                                                                                                      | Cada usuario solo su fila                     |
| <code>signup_requests</code> | <code>id</code> , <code>nombre</code> , <code>apellido</code> , <code>email</code> , <code>username</code> , <code>status</code> , <code>created_at</code>                         | Insertar: público. Ver/gestionar: solo admins |
| <code>profiles</code>        | <code>user_id</code> (pk), <code>email</code> , <code>name</code> , <code>username</code> (único), <code>updated_at</code>                                                         | Leer: autenticados. Escribir: su propia fila  |
| <code>shared_games</code>    | <code>id</code> , <code>owner_email</code> , <code>data</code> (jsonb), <code>members</code> (text[]), <code>rev</code> , <code>updated_by</code> (_name), <code>updated_at</code> | Ver/editar: dueño o miembro. Borrar: dueño    |
| <code>game_invites</code>    | <code>id</code> , <code>game_id</code> , <code>email</code> , <code>inviter_email</code> , <code>game_name</code> , <code>status</code>                                            | Ver/editar: invitado o quien invita           |
| <code>admins</code>          | <code>email</code> (pk)                                                                                                                                                            | Cada uno solo ve su propia fila               |
| <code>announcements</code>   | <code>id</code> , <code>text</code> , <code>created_by</code> , <code>created_at</code>                                                                                            | Leer: autenticados. Publicar/borrar: admins   |

**Funciones** (SECURITY DEFINER): `accept_game_invite(uuid)` (aceptar invitación), `leave_shared_game(uuid)` (salir de un compartido).

El contenido real de cada juego (ideas, reglamento, versiones, prototipo, posts de bitácora) va dentro del campo `data` (**jsonb**) — en `app_state` para los propios y en `shared_games` para los compartidos.

## 9. Seguridad

- La **anon key** de Supabase es pública por diseño (va en el cliente); por eso `config.js` se sube al repo. **Nunca** pongas ahí la `service_role` key.

- La privacidad real la dan **RLS** (cada usuario ve solo lo suyo y lo compartido) y tener **"Allow new users to sign up" desactivado**.
  - **Regla de oro**: no borres en Supabase un usuario que tenga datos — al borrarlo se borran sus datos en cascada. Para cerrar acceso, usa el ajuste de signups, no borres usuarios.
  - Las imágenes de la Bitácora van **incrustadas** (comprimidas) dentro del `data`. Si crecen mucho, conviene migrar a **Supabase Storage**.
- 

## 10. Copias de seguridad

---

- **En la app**: ↓ Exportar / ↑ Importar JSON.
  - **En la base de datos**: ejecutar `backup-app-state.sql` (crea una copia con fecha de `app_state`; no borra nada). Al crear la copia, elige **"Run and enable RLS"** para que quede protegida.
- 

## 11. Trabas conocidas

---

- Al pegar SQL, conserva los `--` de los comentarios (si se pierden, da error de sintaxis).
  - `url` de `config.js` debe ser `https://xxxx.supabase.co` (sin ruta).
  - Correos de invitación con dominio antiguo → revisar **Site URL / Redirect URLs** en Authentication.
  - Un correo que ya existe no se puede "invitar" de nuevo → usar "Send password recovery".
  - El correo integrado de Supabase es lento y limita envíos; puede ir a spam.
- 

## 12. Aviso por email de nuevas solicitudes (opcional)

---

Cuando alguien envía una **solicitud de acceso**, el administrador recibe un **email al instante**. Se consigue sin servidor propio con tres piezas: una **Edge Function** de Supabase (`notify-signup` / el nombre que le pongas) que envía el correo vía **Resend**, y un **disparador** en la base de datos que la llama al insertarse la solicitud. El paso a paso completo está en `edge-notify-signup/GUIA.md`; resumen:

1. **Resend**: crea cuenta (<https://resend.com>), genera una **API Key**. Para empezar puedes usar el remitente de prueba `onboarding@resend.dev`, que **solo envía a tu propio correo** (para enviar desde un remitente propio o a otros destinatarios, verifica un dominio en Resend).
2. **Edge Function**: en Supabase → **Edge Functions** → crear una función y pegar `edge-notify-signup/index.ts`. Despliega con **"Verify JWT" desactivado** (el disparador no manda sesión de usuario; la protección es el `HOOK_SECRET`).
3. **Secrets** (Edge Functions → Secrets): `RESEND_API_KEY`, `ADMIN_EMAIL` (correo donde recibir el aviso), `HOOK_SECRET` (cadena larga aleatoria) y, opcional, `MAIL_FROM`.

4. **Disparador**: como el menú de "Database Webhooks" puede no estar visible, se monta con SQL: `webhook-trigger.sql` (crea la extensión `pg_net`, una función `notify_signup_webhook` que hace `net.http_post` a la URL de la Edge Function con la cabecera `x-hook-secret`, y un trigger `after insert` en `signup_requests`). **Antes de ejecutarlo**, sustituye la URL por la de tu función y pon el mismo `HOOK_SECRET`.

## Probarlo

Inserta una solicitud de prueba (en SQL Editor):

```
insert into public.signup_requests (nombre, apellido, email, username, status)
values ('Prueba', 'Test', 'prueba@ejemplo.com', 'prueba', 'pending');
```

Debe llegarte el email en segundos. Borra luego la fila de prueba. Para diagnosticar: **Edge Functions → Logs** (si la función se ejecutó y la respuesta de Resend). Un **401** en los logs = el `x-hook-secret` del disparador no coincide con el secret `HOOK_SECRET`.

Este aviso **no cambia** el flujo de aprobación: sigues aprobando desde el buzón (✉) y creando el usuario en Supabase. Solo te enteras en el momento.

## Anuncios del administrador

Relacionado: desde el buzón (✉), un administrador puede **publicar anuncios** para todos los usuarios (tabla `announcements`, ver `announcements-schema.sql`). A cada usuario se le "enciende" el icono del buzón cuando hay un anuncio o invitación sin leer.

## 13. Aprobación de usuarios con un clic (Edge Function `approve-signup`)

Automatiza el alta: al pulsar **Aprobar** en el buzón, la app invita al usuario sin que el admin entre a Supabase. Detalle en `edge-approve-signup/GUIA.md`; resumen:

- **Función** `approve-signup` ( `edge-approve-signup/index.ts` ): usa la **service\_role** (secreta, solo servidor). Verifica que quien llama es admin (tabla `admins`) y entonces llama a `auth.admin.inviteUserByEmail(email, { data })`, que envía el correo para fijar contraseña. Si el correo ya existía, hace `resetPasswordForEmail`. Marca la solicitud como *Aprobada*.
- **Despliegue**: crear la función con nombre `approve-signup` y **"Verify JWT" desactivado** (verifica el token ella misma; así el preflight CORS funciona). **No requiere secrets nuevos**: usa los *Default secrets* `SUPABASE_URL` y `SUPABASE_SERVICE_ROLE_KEY`.
- **App**: `Store.approveSignup()` invoca la función con la sesión del admin; el botón **Aprobar** del buzón la usa.
- El **@usuario** solicitado se guarda en los metadatos del usuario ( `requested_username` ) y se pre-rellena en su primer acceso.

- Diagnóstico: **Edge Functions** → **approve-signup** → **Logs** (403 = tu correo no está en `admins`).

## Panel de usuarios activos ( `admin-users` )

El **buzón** (✉) queda solo para notificaciones: **Solicitudes** (pendientes) y **Anuncios**. La lista de **usuarios activos** con su **último acceso** y fecha de alta está en ⚙ **Ajustes** → **pestaña Usuarios** (solo admin). Esa lista la sirve la Edge Function `admin-users` ( `edge-admin-users/index.ts` ): verifica que quien llama es admin y devuelve correo, @usuario, nombre, `created_at` y la última actividad, uniendo `auth.users` con `profiles`. Despliega igual que `approve-signup`: nombre `admin-users`, **Verify JWT off**, sin secrets nuevos.

**Último acceso:** se usa `profiles.updated_at` (que se actualiza en cada arranque de la app mediante `upsertProfile`), **no** `auth.users.last_sign_in_at`. Motivo: `last_sign_in_at` solo cambia en un inicio de sesión real (contraseña/invitación/enlace), no cuando la sesión se refresca sola; `profiles.updated_at` refleja mejor "la última vez que abrió la app". Se usa `last_sign_in_at` solo como respaldo.

Los **anuncios** son solo in-app (no se envían por email).

---

## 14. Personalizar los correos de acceso (remitente y texto)

---

Por defecto los correos de Auth llegan de "Supabase Auth". Para que lleguen como **Kaizen** y con texto propio, ver `email-personalizacion/GUIA.md`. En resumen: el **remitente** se cambia activando **SMTP propio** en Authentication (p. ej. con Resend + dominio verificado, Sender name "Kaizen"); el **texto/diseño** se edita en **Authentication** → **Email Templates** (Invite, Reset Password...), usando variables como `{{ .ConfirmationURL }}`.